Databázy v C#

CRUD operácie a profesionálny Repository Pattern

</> Dnes sa naučíme písať kód, ktorý sa reálne používa v praxi.

Čo je to CRUD?

Základ každej aplikácie (Instagram, TikTok, systémy bánk...). Všetky manipulujú s dátami pomocou štyroch základných operácií.

Písmeno	Operácia (Akcia)	Príkaz v SQL
C	Create (Vytvoriť nový záznam)	`INSERT`
R	Read (Načítať záznamy)	`SELECT`
U	Update (Upraviť existujúci záznam)	`UPDATE`
D	Delete (Vymazať záznam)	`DELETE`

Problém bez organizácie (Chaos)

Čo keby sme nemali Repository?

Bez oddelenia logiky píšeme surové SQL príkazy priamo do vstupného bodu aplikácie (`Program.cs`).

- ✕ **Všetko zmiešané:** SQL sa mieša s výpismi do konzoly a biznis logikou.
- ✕ **Nočná mora pri zmene:** Ak zmeníme názov tabuľky, musíme to prepisovať na desiatkach miest.
- ✕ **Neprehľadnosť:** Súbor náchylný na chyby, ktorý sa rýchlo stane nečitateľným.

```
// Program.cs - absolútny chaos
db.Execute("INSERT INTO products VALUES ('Notebook', 999.99,
5)");

db.Execute("SELECT * FROM products");

db.Execute("DELETE FROM products WHERE id = 1");

db.Execute("INSERT INTO customers VALUES ('Ján',
'jan@gmail.com')");

db.Execute("UPDATE products SET price = 1299 WHERE id = 2");
// SQL je všade, žiadna štruktúra, nebezpečné...
```

Riešenie: Repository Pattern

Základné pravidlo: **Každá tabuľka v databáze má svoju vlastnú triedu.** Každá vrstva komunikuje len s vrstvou pod ňou.

1. Program.cs

Hlavná aplikácia. Obsahuje **len volania C# metód**. Absolútne žiadne SQL príkazy.

2. Repositories

Napríklad `ProductRepository`. Vykonáva konkrétne CRUD operácie (generuje SQL) len pre svoju tabuľku.

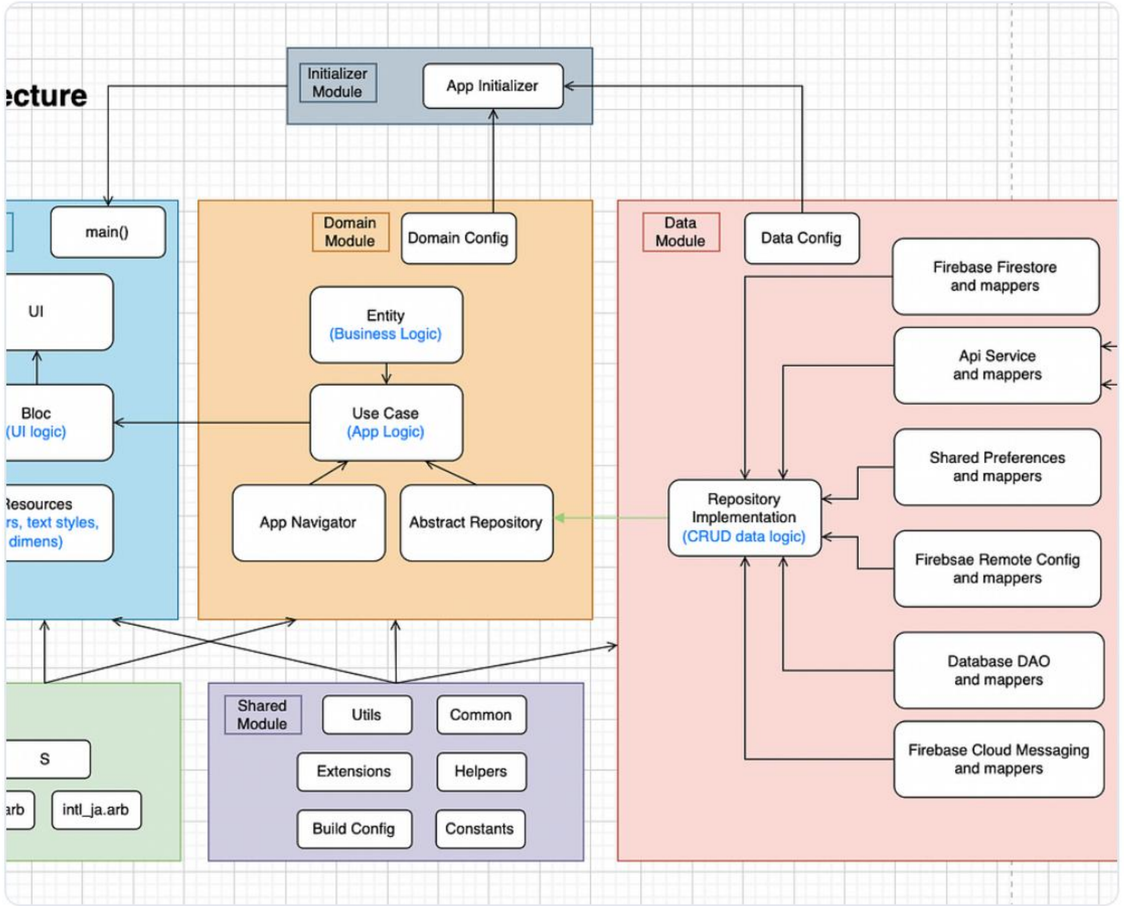
3. Database

Nízkoúrovňová trieda, ktorá spravuje fyzické pripojenie a preposiela príkazy do PostgreSQL.

Benefity Repository Patternu

Tento návrhový vzor je štandardom v softvérovom inžinierstve vďaka svojim obrovským výhodám:

- **Separation of Concerns (SoC)**
  Databázový kód je oddelený. UI vývojár nemusí vidieť a rozumieť SQL.
- **DRY (Don't Repeat Yourself)**
  INSERT dotaz na uloženie produktu napíšeme iba raz a voláme ho všade.
- **Jednoduchá údržba**
  Ak sa zmení názov stĺpca v databáze, upravíme len jedno jediné miesto (Repository triedu).
- **Testovateľnosť**
  Databázu vieme jednoducho "odpojiť" a testovať logiku aplikácie samostatne.



Toto používa každý framework

Tento prístup nie je teoretický. Naučili ste sa základ toho, ako fungujú najpopulárnejšie profesionálne frameworky na trhu.

Java Spring Boot

Využíva modul **Spring Data JPA**. Vytvoríte len rozhranie a framework vygeneruje implementáciu Repository za vás.

C# ASP.NET Core

Používa **Entity Framework Core**. `DbSet` v podstate reprezentuje Repository pre konkrétnu tabuľku vo vašej databáze.

PHP Laravel

Využíva **Eloquent ORM** (Active Record), no v rozsiahlejších Laravel projektoch sa architektúra pre lepšiu správu štandardne balí do vlastných Repositories.

Model trieda (Mapovanie tabuľky)

C# Model: `Product.cs`

Reprezentuje jeden konkrétny riadok z tabuľky `products` v našej aplikácii ako objekt.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int InStock { get; set; }
}
```

PostgreSQL Tabuľka

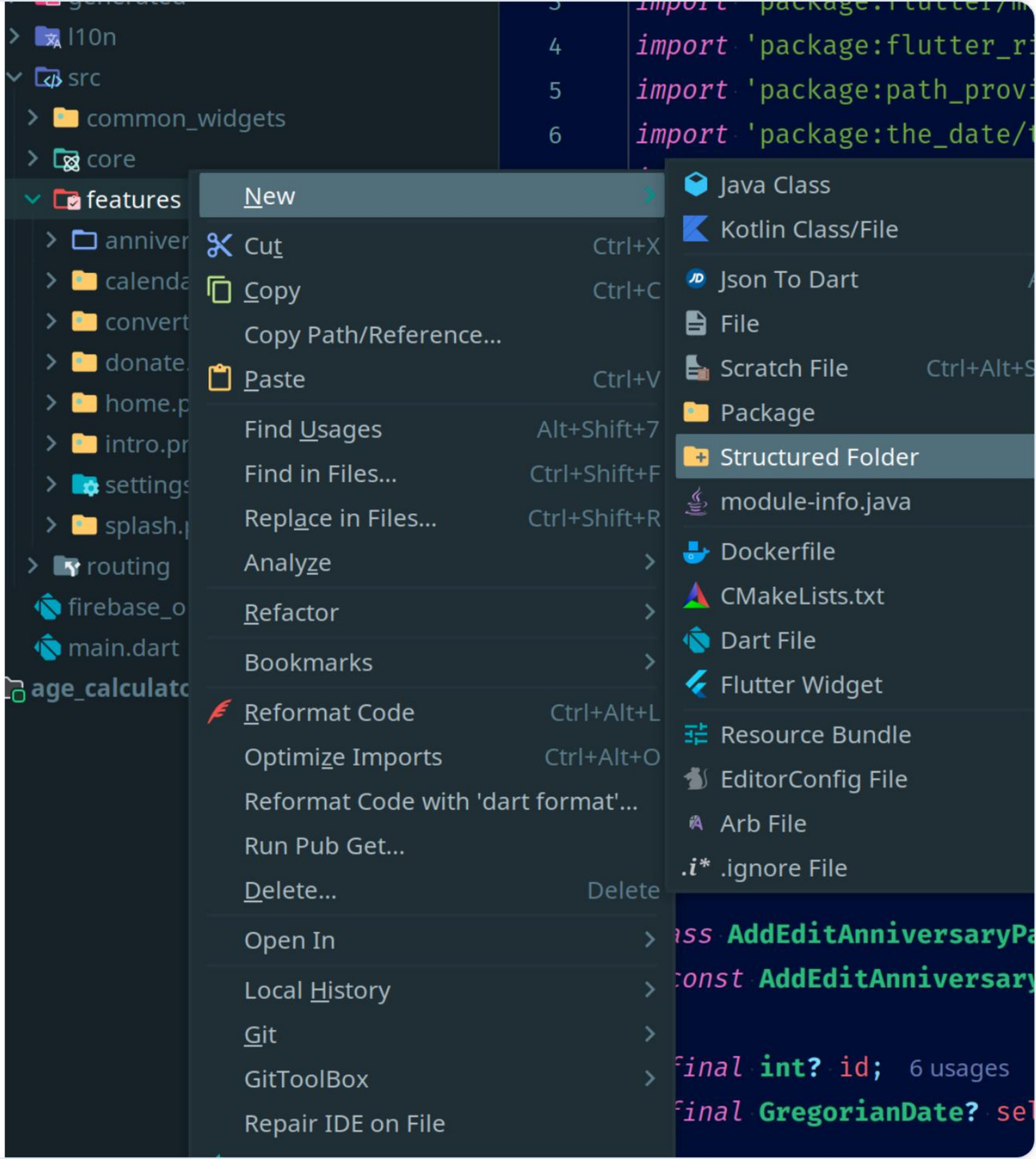
Obojstranné mapovanie dátových typov:

id (INT)	⇄	int Id
name (VARCHAR)	⇄	string Name
price (DECIMAL)	⇄	decimal Price
instock (INT)	⇄	int InStock

Štruktúra projektu

Takto vyzerá čistý organizovaný projekt:

- `EshopApp/` – Root projektu
- `.env` – Zdieľané heslá (nepatrí na GitHub)
- `Program.cs` – Vstupný bod (žiadne SQL)
- `LoadEnv.cs` – Načíta súbor .env
- `Database.cs` – Pripojenie na DB
- `Models/`
  - `Product.cs` – Prepravka pre dáta
- `Repositories/`
  - `ProductRepository.cs` – CRUD pre produkty



C – Create (INSERT)

```
public void Create(Product product)
{
    var price = product.Price.ToString(
        System.Globalization.CultureInfo.InvariantCulture
    );

    _db.Execute($"INSERT INTO products (name, price, instock) " +
                $"VALUES ('{product.Name}', {price}, {product.InStock})");
}
```

Použitie:

```
products.Create(new Product {
    Name = "Notebook",
    Price = 999.99m,
    InStock = 5
});
```

Pozor na InvariantCulture!

> Slovenský počítač formátuje čísla s **čiarkou** (999,99). PostgreSQL ale vyžaduje anglickú **bodku** (999.99).
>
> `InvariantCulture` vynúti, aby sa vždy použila bodka, bez ohľadu na to, aký jazyk je nastavený vo Windows/Mac.

R – Read (SELECT)

```
public List<Product> GetAll()
{
    var reader = _db.Query("SELECT * FROM products");
    var products = new List<Product>();

    while (reader.Read())  // prechádza riadok po riadku
    {
        products.Add(new Product
        {
            Id = reader.GetInt32(0),
            Name = reader.GetString(1),
            Price = reader.GetDecimal(2),
            InStock = reader.GetInt32(3)
        });
    }
    reader.Close();
    return products;
}
```

Ako to funguje?

- Cyklus `while (reader.Read())` sa posúva po výsledkoch databázy riadok po riadku.
- Čísla **0, 1, 2, 3** reprezentujú poradie stĺpcov v tabuľke (počítané od nuly).
- Metóda vracia "čistý" `List` C# objektov. Volajúci (Program.cs) s databázovým readerom nepríde do styku.

U – Update a D – Delete

```
// UPDATE
public void Update(int id, Product product)
{
    var price = product.Price.ToString(
        CultureInfo.InvariantCulture
    );
    _db.Execute($"UPDATE products SET name = '{product.Name}', "
+
                $"price = {price}, instock = {product.InStock} "
+
                $"WHERE id = {id}");
}

// DELETE
public void Delete(int id)
{
    _db.Execute($"DELETE FROM products WHERE id = {id}");
}
```

Použitie metód:

```
// Úprava produktu s ID = 1
products.Update(1, new Product {
    Name = "Notebook Pro",
    Price = 1299.99m,
    InStock = 3
});

// Vymazanie produktu s ID = 2
products.Delete(2);
```

Výsledok – čistý Program.cs

```
LoadEnv.LoadFile(".env");
var db = new Database();
var products = new ProductRepository(db);

// Vytvorenie
products.Create(new Product { Name = "Notebook", Price = 999.99m, InStock = 5
});

// Výpis všetkých
foreach (var p in products.GetAll())
    Console.WriteLine($"{p.Name} | {p.Price}€");

// Úprava a Vymazanie
products.Update(1, new Product { Name = "Notebook Pro", Price = 1299m,
InStock = 3 });
products.Delete(2);

db.Close();
```

Kde je SQL kód?

> **Presne tak, nikde!** Toto je cieľ Repository patternu.

Váš hlavný kód sa stal plne objektovým. Je čitateľný zhora nadol ako kniha a o všetku špinavú prácu s databázou sa stará vrstva pod ním.

Tvoja dnešná úloha

Vyskúšajte si architektúru, ktorú stretnete vo svojej prvej programátorskej práci.

1. **Vytvor Model triedu**

   V zložke `Models/` vytvor triedu `Customer.cs`, ktorá mapuje SQL tabuľku zákazníkov (Napr. Id, Name, Email).

2. **Vytvor Repository triedu**

   V zložke `Repositories/` vytvor `CustomerRepository.cs`. Naimplementuj všetky 4 CRUD operácie (Create, GetAll, Update, Delete).

   Hint: `ProductRepository` je tvoj dokonalý vzor. Štruktúra je identická.

3. **Otestuj to naostro**

   Prejdi do `Program.cs`, vytvor inštanciu `CustomerRepository` a pomocou C# metód vytvor, vypíš, uprav a vymaž zákazníka. Nakoniec si to vizuálne skontroluj v PostgreSQL.

Image Sources

https://miro.medium.com/v2/resize:fit:1400/1*C2Euk2Dc9H3Pyf7I9WY3cw.png

Source: medium.com

---

https://upload.wikimedia.org/wikipedia/commons/thumb/4/44/Spring_Framework_Logo_2018.svg/1280px-Spring_Framework_Logo_2018.svg.png

Source: ru.wikipedia.org

---

https://cdn.dribbble.com/userupload/21359917/file/original-6949a8f0c08103ef18a6dd8bb6f15b73.jpg

Source: dribbble.com

---

https://picperf.io/https://laravelnews.s3.amazonaws.com/images/laravel-featured.png

Source: laravel-news.com

---

https://plugins.jetbrains.com/files/26496/screenshot_2b3f6bb5-46e7-4576-b3ec-29ceede33bd2

Source: plugins.jetbrains.com